

Oprogramowanie Systemowe

Linux – process scheduling

mgr inż. Marek Downar



Scheduler

- Odpowiada za przekazywanie zasobów procesora pomiędzy procesami.
- Zabezpiecza przed zmonopolizowaniem procesora przez jeden proces.
- Do kernela 2.4 scheduler był prosty w założeniach, ale słabo skalował wiele procesów,
- W wersjach 2.5, 2.6 wprowadzono nowe wersje schedulerów - $O(1)$ scheduler, staircase deadline oraz completely fair scheduling



I/O vs Processor

- Procesy mogą być powiązane z procesorem bądź z operacjami I/O
- Procesy powiązane z procesorem wykonywane są rzadziej lecz przez dłuższy czas.
- Trzeba zaspokoić dwa sprzeczne cele: szybka odpowiedź procesu (low latency) oraz maksymalne wykorzystanie zasobów systemowych (high throughput).
- Linux preferuje procesy powiązane z operacjami I/O. Celem jest udostępnianie wysokiej interaktywności.



Planowanie priorytetowe

- Standardowy scheduler jest algorytmem opartym o priorytety. Procesom nadane muszą być priorytety i ich zapotrzebowanie na czas procesora. Procesy o wyższym priorytecie uruchamiane są przed procesami o niższym priorytecie. Procesy o tym samym priorytecie planowane są przy pomocy algorytmu *round-robin*.



Priorytety - Linux

- *nice* (-20:+19) - domyślnie 0 – kontrola proporcji przydzielonej jednostki procesora
- *rtp* – (*real-time priority*) (0:99)
ps -eo state,uid,pid,ppid,rtprio,time,comm



Jednostka czasu (*timeslice*)

- *Timeslice* – wartość określająca, jak długo zadanie może być wykonywane dopóki nie zostanie wywłaszczone. Polityka schedulera określa domyślną wartość.
- Procesy I/O nie potrzebują dużych *timeslice* (uruchamiane są często), procesy powiązane z procesorem natomiast wymagają dłuższych jednostek czasu (w celu efektywnego wykorzystania elementów cachowanych).



Klasy schedulera

- W linux planista jest modularny, możliwe jest uruchomienie różnych algorytmów w celu planowania różnych typów procesów. Taka modularność nazywana jest *scheduler classes*.
- Każda klasa schedulera posiada swój priorytet.
- Podstawowy kod schedulera <kernel/sched.c> iteruje po klasach schedulera w kolejności priorytetów.
- Klasa o najwyższym priorytecie, wg której zaplanowany został aktualnie wykonywany proces decyduje o kolejnym procesie.



Problemy

- Bezpośrednie odwzorowanie *timeslice* na wartość *nice*
- Relatywne odwzorowanie
- Absolutne przypisanie
- Optymalizacja dla procesów interaktywnych



CFS

- Modelowanie planowania procesów tak, jakby system posiadał idealny wielozadaniowy procesor (każdy proces otrzymuje $1/n$ czasu procesora). Planowanie z nieskończenie małym okresem.
- CFS – oblicza jak długo proces powinien pracować na podstawie wszystkich procesów.
- *nice* wykorzystywany jest w celu wyważenia procesu – przydzielenia jemu określonej proporcji procesora (wyższy priorytet – większa część procesora)
- Każdy proces działa przez *timeslice* proporcjonalne do swojej wagi podzielonej przez wagi wszystkich procesów.
- Istotna jest tylko relatywna różnica pomiędzy wartościami *nice* (różnica geometryczna)
- CFS – ustala cel „nieskończenie małego czasu” (*targeted latency*) – mniejszy cel – lepsza interaktywność (lepsze *latency* słabsze *throughput*). Przykład: *targeted latency* – 20 ms – 2 procesy – każdy po 10ms, 20 procesów – każdy po 1ms
- *Minimalna granularność* – domyślnie 1ms
- *Przykład:*
 - 2 procesy – jeden nice=0 drugi nice=5, targeted latency = 20.*
 - Proces o nice=0 otrzymuje 15ms, proces o nice 5 otrzymuje 5 ms*
 - 2 procesy – jeden nice=10 drugi nice =15 ?*

